



Katedra informatiky a výpočetní techniky

Semestrální práce z předmětu
Paralelní programování

Simulace měření kvantového stavu

Autor:

Bc. Stanislav Kafara

A24N0088P

skafara@students.zcu.cz

Obsah

1	Zadání	3
2	Teoretické pozadí	4
2.1	Kvantový výpočet	4
2.2	Simulace měření	4
3	Návrh řešení	6
3.1	Použité technologie	6
3.2	Schéma řešení	6
3.3	Vzorkování z pravděpodobnostní distribuce	7
3.4	Generování náhodných čísel	8
4	Popis implementace	9
4.1	Utility	9
4.1.1	Zarovnání paměti	9
4.1.2	Šablonované operace	10
4.1.3	OpenCL wrapper	11
4.2	Simulace měření	12
4.2.1	Vstupní bod programu	12
4.2.2	Odstranění prokládání	13
4.2.3	Výpočet pravděpodobností	13
4.2.4	Sestavení Alias tabulky	16
4.2.5	Vzorkování z Alias tabulky	16
4.2.6	Generování pseudonáhodné sekvence	18
4.2.7	Zápis vzorků do výstupního pole	21
5	Výsledky experimentů	22
5.1	Robustnost Philox PRNG	22
5.2	Urychlení výpočtů	23
5.3	Poznámky	24

6	Uživatelská příručka	26
6.1	Překlad a sestavení	26
6.2	Spuštění	27
7	Závěr	28
A	Statistická podobnost distribucí naměřených výsledků	33
B	Urychlení výpočetního času	36
C	Naměřený výpočetní čas	40

Kapitola 1

Zadání

Cílem semestrální práce je navrhnout a implementovat simulaci měření stavu kvantového výpočtu. Výstupem bude program se třemi způsoby výpočtu: sekvenční skalární, vícevláknový s manuální vektorizací (**AVX2**) a paralelní s využitím GPU. Řešení bude realizováno jako dynamicky linkovaná knihovna (DLL) k existující implementaci simulátoru kvantového počítače. Výsledná implementace bude otestována na správnost, bude porovnána rychlost implementovaných výpočtů a robustnost výsledků v porovnání se skutečně náhodnou simulací.

Kapitola 2

Teoretické pozadí

2.1 Kvantový výpočet

Kvantový výpočet představuje výpočetní paradigma založené na principech kvantové mechaniky, které se v zásadních aspektech liší od klasického pojetí výpočtu. Zatímco klasické počítače pracují s bity nabývajících hodnot 0 nebo 1, kvantové počítače využívají kvantové bity (qubity), jejichž stav může být popsán jako lineární kombinace obou těchto hodnot. Tato vlastnost, označovaná jako superpozice, umožňuje kvantovým systémům reprezentovat více stavů současně.

Výpočet v kvantovém počítači je realizován postupnou aplikací kvantových hradel, která odpovídají unitárním transformacím kvantového stavu. Výsledkem kvantového výpočtu není přímo konkrétní hodnota, ale kvantový stav, ze kterého je teprve pomocí měření získána klasická informace. Vzhledem k tomu, že kvantové systémy podléhají pravděpodobnostnímu chování, je výsledek měření obecně nedeterministický. Praktické využití kvantových algoritmů proto vyžaduje opakované provádění výpočtu a statistické vyhodnocení naměřených výsledků.

2.2 Simulace měření

Měření kvantového stavu je finální část kvantového výpočtu, při kterém je kvantová informace převedena na klasický výsledek. Kvantový stav systému je popsán stavovým vektorem, jehož jednotlivé složky odpovídají amplitudám pravděpodobností měřitelných stavů. Pravděpodobnost získání konkrétního výsledku měření je dána druhou mocninou absolutní hodnoty příslušné amplitudy.

V případě simulace se jedná o numerickou operaci nad stavovým vektorem.

rem. Simulace měření spočívá ve výpočtu pravděpodobnostního rozdělení všech možných výsledků a následném náhodném výběru jednoho z nich. Tento výběr je typicky realizován pomocí generátoru pseudonáhodných čísel, přičemž výsledná hodnota je v naivním případě určena porovnáním generovaného náhodného čísla s kumulativní distribuční funkcí odpovídající danému kvantovému stavu.

Kapitola 3

Návrh řešení

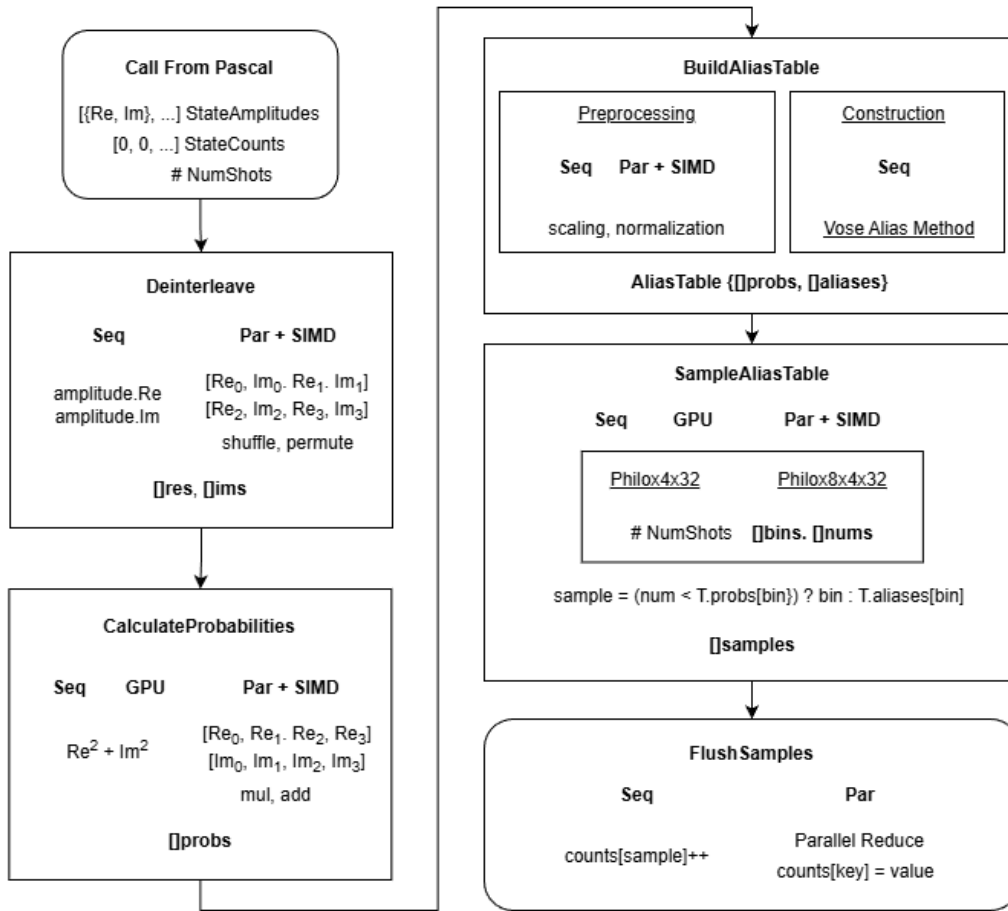
3.1 Použité technologie

Řešení bude realizováno v programovacím jazyce C++ jako dynamicky linkovaná knihovna (DLL) k existující implementaci simulátoru kvantového počítače. Výpočty na GPU budou realizovány pomocí OpenCL. Vektorizace bude realizována manuálně pomocí intrinsics podle standardu AVX2. Paralelizace bude realizována pomocí funkcí standardní knihovny a knihovny oneTBB. Jednotlivé implementace způsobů výpočtu budou realizovány pomocí specializací šablonových funkcí.

3.2 Schéma řešení

Řešení je vizualizováno v diagramu 3.1 a skládá se z šesti hlavních kroků:

1. zavolání vstupního bodu programu ze simulátoru,
2. předzpracování obdržených dat,
3. výpočet pravděpodobnostní distribuce,
4. předzpracování pravděpodobnostní distribuce,
5. vzorkování z pravděpodobnostní distribuce a
6. zápis vzorků do výstupního pole.



Obrázek 3.1: Diagram řešení

3.3 Vzorkování z pravděpodobnostní distribuce

Vzhledem k velikosti stavového prostoru kvantového výpočtu, která roste exponenciálně s počtem qubitů, je efektivní realizace tohoto kroku zásadní z hlediska výpočetní náročnosti celé simulace.

Naivní přístup

V naivním přístupu je vzorkování realizováno porovnáním náhodně vygenerované hodnoty s kumulativní distribuční funkcí, což vyžaduje lineární průchod polem pravděpodobností. Tento postup má časovou složitost $\mathcal{O}(n)$ na jeden vzorek, kde n odpovídá počtu možných měřitelných stavů.

Voseova aliasová metoda

Z výše zmíněných důvodů je v rámci navrženého řešení použita *Voseova aliasová metoda* [1], která umožňuje vzorkování z diskrétní pravděpodobnostní distribuce v konstantním čase $\mathcal{O}(1)$ po jednorázovém předzpracování distribuce s lineární složitostí $\mathcal{O}(n)$. Metoda spočívá v transformaci původní distribuce do dvou pomocných polí, která pro každý stav uchovávají pravděpodobnostní práh a případný aliasový index.

Během samotného vzorkování je následně náhodně zvolen index a pomocí další náhodné hodnoty je rozhodnuto, zda je vybrán přímo odpovídající stav, nebo jeho alias. Tento postup je dobře paralelizovatelný a vhodný jak pro vektorové zpracování na CPU, tak pro implementaci na GPU.

3.4 Generování náhodných čísel

Proces vzorkování z pravděpodobnostní distribuce je inherentně založen na náhodném výběru. Kvalita použitého zdroje náhody má přímý vliv na statistické vlastnosti výsledků simulace.

Důležitost skutečné náhody

Nevhodně zvolený generátor pseudonáhodných čísel může vést ke vzniku korelací, systematických odchylek a zkreslení výsledné distribuce naměřených výsledků. Za dobrý zdroj náhody můžeme považovat např. službu *Random.org* [2], která poskytuje zdroj skutečně náhodných čísel odvozených z naměřených atmosférických dat. Zdroj skutečné náhody je ale výrazně obtížné zajistit, natož paralelizovat. Z toho důvodu se v praxi často používají generátory pseudonáhodných čísel.

Philox PRNG

V rámci navrženého řešení je použit pseudonáhodný generátor *Philox* [3], který patří do třídy tzv. *counter-based* generátorů. Na rozdíl od tradičních stavových generátorů, jako je například Mersenne Twister, neudrží *Philox* vnitřní stav závislý na předchozích generovaných hodnotách. Každá pseudonáhodná hodnota je deterministicky vypočtena jako funkce čítače a klíče.

Tato vlastnost činí generátor *Philox* zvláště vhodným pro paralelní výpočty. V prostředí GPU výpočtu nebo vícevláknového zpracování lze každému vláknu přiřadit nezávislou část prostoru čítače bez jakékoliv přídavné režie synchronizace. Zároveň je možné generovat větší bloky náhodných čísel najednou a algoritmus efektivně implementovat vektorově.

Kapitola 4

Popis implementace

Program je rozdělen do modulů `KQS.Simulator`, `KQS.Random`, `KQS.Complex`, `KQS.Simulator`, `KQS.CLManager` a `KQS.Utills`, kde každý obstarává odpovídající funkcionalitu.

4.1 Utility

4.1.1 Zarovnání paměti

Pro efektivní implementaci vektorového výpočtu programu je potřeba mít data zarovnaná na velikost strojového slova.

Dynamická alokace paměti

Pro snadnou práci s dynamickou alokací zarovnané paměti byl implementován `AlignedAllocator<T, Alignment>`, který alokuje paměť zarovnanou na `Alignment` bytů. `AlignedVector<T, Alignment>` je pak klasický knihovní `std::vector<T>`, který alokuje paměť zarovnanou na `Alignment` bytů. Implementace využívá `AlignedVector64`.

```
template <typename T, std::size_t Alignment>
using AlignedVector = std::vector<T, AlignedAllocator<T,
    Alignment>>;

template <typename T>
using AlignedVector64 = AlignedVector<T, 64>;
```

Zdrojový kód 4.1: Dynamická alokace zarovnané paměti

Statická alokace paměti

Pro statickou alokaci zarovnané paměti se používá `std::array<T, N>` s klíčovým slovem `alignas(M)`. Implementace využívá 64 bytové zarovnání, tedy `alignas(64) std::array<T, N>`.

4.1.2 Šablonované operace

Pro snadnou implementaci všech třech větví výpočtu v jednom programu jsou jednotlivé operace, které jsou implementačně rozdílné pro tyto tři způsoby výpočtu, šablonové. Jednotlivé způsoby výpočtu a implementace generátorů náhodných čísel jsou jejich specializace.

```
enum class ExecutionPolicy {  
    /** @brief Sequential execution */  
    Sequential,  
    /** @brief Parallel + SIMD execution on CPU */  
    Parallel,  
    /** @brief OpenCL accelerated execution on GPU */  
    Accelerated  
};
```

Zdrojový kód 4.2: Výčet způsobů výpočtu

```
enum class PrngAlgorithm {  
    /** @brief Philox algorithm */  
    Philox,  
    /** @brief Mersenne Twister 19937 algorithm */  
    MT19937,  
    /** @brief Random.org historical random data */  
    RandomOrg  
};
```

Zdrojový kód 4.3: Výčet generátorů náhodných čísel

Kontejner specifický pro výpočetní zařízení

Pro snadnou definici jednotného rozhraní programu a předávání dat z a do jednotlivých funkcí implementujících výpočty byl implementován šablonový kontejner, nad kterým jednotlivé funkce operují. Operace implementované na CPU tak budou pracovat se zarovnanou pamětí v RAM a operace implementované na GPU budou pracovat s pamětí alokovanou na GPU a budou přitom sdílet shodné rozhraní.

```

template <ExecutionPolicy Policy, typename T>
struct DeviceContainer {
    /** @brief Type of container */
    using type = AlignedVector64<T>;
    /** @brief Reference type */
    using ref_type = std::span<T>;
    /** @brief Constant reference type */
    using ref_const_type = std::span<const T>;
};

template <typename T>
struct DeviceContainer<ExecutionPolicy::Accelerated, T>
{
    using type = cl::Buffer;
    using ref_type = type &;
    using ref_const_type = const type &;
};

```

Zdrojový kód 4.4: Kontejner specifický pro výpočetní zařízení

4.1.3 OpenCL wrapper

Třída CLManager představuje wrapper pro užitečnou práci s OpenCL knihovnou. Třída umožňuje načíst programy z několika .cl souborů, nastavit prostředí a provádět užitečné opakované zaobalené paměťové i výpočetní operace nad OpenCL zařízením.

```

template <typename T>
static inline cl::Buffer AllocateWriteOnly(size_t size);

template <typename T>
static inline cl::Buffer AllocateFromReadOnly(std::span<
    const T> data);

template <typename... Args>
static inline cl::Event Launch1D(std::string kernelName,
    size_t globalSize, Args&&... args);

template <typename T>
static inline void ReadbackBlocking(const cl::Buffer&
    buffer, std::span<T> out);

```

Zdrojový kód 4.5: Ukázka rozhraní CLManager

4.2 Simulace měření

4.2.1 Vstupní bod programu

Vstupním bodem je vystavená funkce `ESimulator_Run`, která přijme výstupní pole, stavový vektor, počet stavů a počet měření. Výstupní pole představuje histogram naměřených hodnot. Funkce inicializuje prostředí a pomocí funkce `Run` postupně volá jednotlivé specializace šablonových funkcí podle zvoleného způsobu výpočtu.

```
extern "C" __declspec(dllexport)
void __cdecl ESimulator_Run(
    uint* AStateCounts,
    const LComplex* AStateAmplitudes,
    uint ANumStates,
    uint ANumShots
);
```

Zdrojový kód 4.6: Rozhraní vstupního bodu programu

```
template
<ExecutionPolicy Policy, PrngAlgorithm Algorithm>
inline void Run(
    std::span<uint> StateCounts,
    std::span<const LComplex> StateAmplitudes,
    const uint NumShots
) {
    // Deinterleave complex amplitudes into real and
    // imaginary parts
    const auto [res, ims] = DeinterleaveAoSLComplex<
        Policy>(StateAmplitudes);
    // Calculate probabilities from amplitudes
    const auto probs = CalculateProbabilities<Policy>(
        res, ims);
    // Build alias table
    const auto table = BuildAliasTable<Policy>(probs);
    // Sample from alias table
    auto samples = SampleAliasTable<Policy, Algorithm>(
        table, NumShots);
    // Flush samples into state counts
    FlushSamples<Policy>(StateCounts, samples);
}
```

Zdrojový kód 4.7: Běh simulace

4.2.2 Odstranění prokládání

Pro efektivní implementaci vektorizovaného výpočtu pravděpodobnosti je vhodné pracovat s jednotlivými složkami komplexního čísla zvlášť. Je potřeba tyto dvě složky oddělit, protože simulátor poskytuje statový vektor prokládaný.

Par

Program zpracovává pole komplexních čísel po čtyřech číslech najednou, tj. osm double čísel najednou. Načte je do dvou vektorových registrů a vektorovými instrukcemi prokládání odstraní.

```
const auto idxes = std::views::iota(0, arr.size() / 4) |
    std::views::transform([] (size_t i) { return i * 4; });
std::for_each(std::execution::par,
    idxes.begin(), idxes.end(),
    [&] (size_t i) {
        // [Re0, Im0, Re1, Im1]
        const __m256d c12 = _mm256_load_pd(reinterpret_cast<
            const double*>(&arr[i]));
        // [Re2, Im2, Re3, Im3]
        const __m256d c34 = _mm256_load_pd(reinterpret_cast<
            const double*>(&arr[i + 2]));
        // [Re0, Re2, Re1, Re3]
        __m256d re_ = _mm256_shuffle_pd(c12, c34, 0b0000);
        // [Im0, Im2, Im1, Im3]
        __m256d im_ = _mm256_shuffle_pd(c12, c34, 0b1111);
        // 0, 2, 1, 3 -> [Re0, Re1, Re2, Re3]
        re_ = _mm256_permute4x64_pd(re_, 0b11011000);
        // 0, 2, 1, 3 -> [Im0, Im1, Im2, Im3]
        im_ = _mm256_permute4x64_pd(im_, 0b11011000);
        _mm256_stream_pd(&res[i], re_);
        _mm256_stream_pd(&ims[i], im_);
    }
);
```

Zdrojový kód 4.8: (Par) Odstranění prokládání

4.2.3 Výpočet pravděpodobností

Komplexní amplitudy se hromadně převedou na pravděpodobnosti odpovídajících stavů podle vzorce $P(i) = |c_i|^2$. Pro ukázkou rozhraní a implementace jsou představeny všechny tři verze.

Seq

```
const auto idxes = std::views::iota(0, res.size());
std::for_each(std::execution::seq,
    idxes.begin(), idxes.end(),
    [&] (size_t i) {
        probs[i] = CalculateProbability(res[i], ims[i]);
    }
);

...

inline double
CalculateProbability(const double re, const double im) {
    return re * re + im * im;
}
```

Zdrojový kód 4.9: (Seq) Výpočet pravděpodobností

Par

Program zpracovává pole obou složek komplexních čísel najednou, každé po čtyřech číslech, tj. tedy dohromady 8 double čísel. Načte je do dvou vektorových registrů a vektorovými instrukcemi spočítá pravděpodobnosti odpovídajících stavů.

```
const auto idxes = std::views::iota(0, res.size() / 4) |
    std::views::transform([] (size_t i) { return i * 4; });
std::for_each(std::execution::par,
    idxes.begin(), idxes.end(),
    [&] (size_t i) {
        // [Re0, Re1, Re2, Re3]
        const __m256d re = _mm256_load_pd(&res[i]);
        // [Im0, Im1, Im2, Im3]
        const __m256d im = _mm256_load_pd(&ims[i]);
        // [Re0^2 + Im0^2, ..., Re3^2 + Im3^2]
        const __m256d prob = CalculateProbability(re, im);

        // Streaming store to avoid cache pollution
        _mm256_stream_pd(&probs[i], prob);
    }
);

...
```

```

inline __m256d
CalculateProbability(const __m256d re, const __m256d im)
{
    // [Re0^2, Re1^2, Re2^2, Re3^2]
    const __m256d re2 = _mm256_mul_pd(re, re);
    // [Im0^2, Im1^2, Im2^2, Im3^2]
    const __m256d im2 = _mm256_mul_pd(im, im);
    // [Re0^2 + Im0^2, ..., Re3^2 + Im3^2]
    const __m256d prob = _mm256_add_pd(re2, im2);
    return prob;
}

```

Zdrojový kód 4.10: (Par) Výpočet pravděpodobností

GPU

Program alokuje buffer vypočtených pravděpodobností na GPU zařízení a zavolá spuštění kernelu. Po dokončení výpočtu se překopírují vypočtené pravděpodobnosti na hostitelské zařízení. Každé vlákno zpracovává jedno číslo.

```

cl::Buffer probsBuffer = CLManager::AllocateWriteOnly<
    double>(probs.size());
CLManager::Launch1D("_CalculateProbabilities", probs.
    size(), res, ims, probsBuffer);
CLManager::ReadbackBlocking(probsBuffer, probs);

...

inline double
CalculateProbability(const double re, const double im) {
    // p = re^2 + im^2
    return re * re + im * im;
}

__kernel void
_CalculateProbabilities(__global const double *res,
    __global const double *ims, __global double *probs) {
    const size_t i = get_global_id(0);

    probs[i] = CalculateProbability(res[i], ims[i]);
}

```

Zdrojový kód 4.11: (GPU) Výpočet pravděpodobností

4.2.4 Sestavení Alias tabulky

Alias tabulka je sestavena z principu algoritmu sekvenčně podle [1]. Výstupem algoritmu jsou dvě pole stejné délky, kde první obsahuje upravené pravděpodobnosti pro výběr původního indexu a druhé uchovává indexy alternativních výsledků.

4.2.5 Vzorkování z Alias tabulky

Vzorkování z Alias tabulky probíhá hromadně tak, že se nejprve vygenerují všechna náhodná čísla potřebná pro vzorkování z tabulky a ty se následně hromadně zpracují podle [1].

```
template
<ExecutionPolicy Policy, PrngAlgorithm Algorithm>
AlignedVector64<uint32>
SampleAliasTable(const AliasTable &table, const uint
    NumShots) {
    // Output samples
    AlignedVector64<uint32> samples(NumShots);

    // Generate random bins and random numbers
    const auto r_bins = GenerateRandomDiscrete<Policy,
        Algorithm>(SeedRandomDiscrete, NumShots, table.
        Probs.size());
    const auto r_rands = GenerateRandomContinuous<Policy,
        Algorithm>(SeedRandomContinuous, NumShots);

    // Sample from alias table
    _SampleAliasTable<Policy>(table, r_bins, r_rands,
        samples);
    return samples;
}
```

Zdrojový kód 4.12: Vzorkování z Alias tabulky

Seq

Algoritmus vzorkování sekvenčně zpracovává dvojice náhodných čísel a podle Alias tabulky rozhoduje, který vzorek je vybrán.

```

const auto idxes = std::views::iota(0, bins.size());
std::for_each(std::execution::seq,
    idxes.begin(), idxes.end(),
    [&] (size_t i) {
        const uint32 bin = bins[i];
        const double rand = rands[i];
        samples[i] = (rand < table.Probs[bin]) ? bin :
            table.Aliases[bin];
    }
);

```

Zdrojový kód 4.13: (Seq) Vzorkování z Alias tabulky

Par

Algoritmus vzorkování paralelně zpracovává bloky dvojic čtyř náhodných čísel. Načte je do vektorového registru a zpracovává je pomocí vektorových instrukcí. Pro vektorové porovnání hodnot se používá maska.

```

...

const __m256d maskV = _mm256_cmp_pd(randsV, probsV,
    _CMP_LT_OQ);
const int mask = _mm256_movemask_pd(maskV);

samples[offset+0] = (mask&0x1) ? binsA[0] : aliasesA[0];
samples[offset+1] = (mask&0x2) ? binsA[1] : aliasesA[1];
samples[offset+2] = (mask&0x4) ? binsA[2] : aliasesA[2];
samples[offset+3] = (mask&0x8) ? binsA[3] : aliasesA[3];

```

Zdrojový kód 4.14: (Par) Vzorkování z Alias tabulky

GPU

Program alokuje potřebné buffery na GPU zařízení, překopíruje data a zavolá spuštění kernelu. Po dokončení výpočtu se překopírují vzorky na hostitelské zařízení. Každé vlákno zpracovává dvojici náhodných čísel a výsledkem je jeden vzorek. Větvení programu je zabráněno pomocí aritmetického výrazu.

```

cl::Buffer probsBuffer = CLManager::
    AllocateFromReadOnly(table.Probs);
cl::Buffer aliasesBuffer = CLManager::
    AllocateFromReadOnly(table.Aliases);

```

```

cl::Buffer samplesBuffer = CLManager::AllocateWriteOnly<
    uint32>(samples.size());

CLManager::Launch1D("_SampleAliasTable", samples.size(),
    probsBuffer, aliasesBuffer, bins, rands,
    samplesBuffer);

CLManager::ReadbackBlocking(samplesBuffer, samples);

...

__kernel void
_SampleAliasTable(
    __global const double *probs,
    __global const uint32 *aliases,
    __global const uint32 *bins,
    __global const double *rands,
    __global uint32 *samples
) {
    const size_t i = get_global_id(0);

    // Branchless sample selection
    const uint32 cond = rands[i] < probs[bins[i]];
    // if rand < prob[bin] then sample = bin else sample
    //   = alias[bin]
    samples[i] = aliases[bins[i]] + cond * (bins[i] -
        aliases[bins[i]]);
}

```

Zdrojový kód 4.15: (GPU) Vzorkování z Alias tabulky

4.2.6 Generování pseudonáhodné sekvence

Algoritmus Philox pro generování pseudonáhodných čísel je implementován podle [3]. Jeho implementace pro výpočet na GPU je zobrazena ve Zdrojovém kódu 4.16. Jeden výpočet Philox vygeneruje čtyři 32 bitové čísla.

V případě sekvenčního výpočtu je implementace téměř identická s implementací pro GPU. V případě paralelního vektorizovaného výpočtu se využije toho, že algoritmus je velmi prostý a může probíhat paralelně vláknově v rámci bloků a paralelně v rámci SIMD linek. Každé vlákno tedy zpracovává určitou část a zároveň jedno vlákno zpracovává osm Philox výpočtů, každý v jedné SIMD lince.

```

inline void
GeneratePhilox4x32_10(const uint64 key, const uint64
    counter, __private uint32 out[4]) {
    const uint64 M0 = 0xD2511F53UL;
    const uint64 M1 = 0xCD9E8D57UL;
    const uint32 W0 = 0x9E3779B9UL;
    const uint32 W1 = 0xBB67AE85UL;

    uint32 k0 = (uint32) key;
    uint32 k1 = (uint32) (key >> 32);

    uint32 x0 = (uint32) counter;
    uint32 x1 = (uint32) (counter >> 32);
    uint32 x2 = 0u;
    uint32 x3 = 0u;

    for (int round = 0; round < 10; ++round) {
        const uint64 p0 = M0 * x0;
        const uint64 p1 = M1 * x2;

        const uint32 hi0 = (uint32) (p0 >> 32);
        const uint32 hi1 = (uint32) (p1 >> 32);
        const uint32 lo0 = (uint32) p0;
        const uint32 lo1 = (uint32) p1;

        const uint32 y0 = hi1 ^ x1 ^ k0;
        const uint32 y1 = lo1;
        const uint32 y2 = hi0 ^ x3 ^ k1;
        const uint32 y3 = lo0;

        x0 = y0;
        x1 = y1;
        x2 = y2;
        x3 = y3;

        k0 += W0;
        k1 += W1;
    }

    out[0] = x0; out[1] = x1; out[2] = x2; out[3] = x3;
}

```

Zdrojový kód 4.16: (GPU) Philox PRNG

Generování diskrétních čísel

Pro vygenerování n (n dělitelné 4) čísel datového typu `uint32_t` je potřeba alokovat paměť o velikosti n čísel datového typu `uint32_t`. Následně se $n/4$ krát provede výpočet Philox se shodným klíčem a inkrementujícím se čítačem. V případě GPU se velikost alokované paměti zaokrouhlí nahoru na nejbližší vyšší čtyřnásobek n a čítač je globální index. Konečně, vygenerované čísla se transformují do požadovaného intervalu $[0, \text{max})$, jak je ukázáno ve Zdrojovém kódu 4.17.

Generování spojitých čísel

Pro vygenerování n (n dělitelné 2) čísel datového typu `double` je potřeba alokovat paměť o velikosti n čísel datového typu `double`. Následně se $n/2$ krát provede výpočet Philox se shodným klíčem a inkrementujícím se čítačem. V případě GPU se velikost alokované paměti zaokrouhlí nahoru na nejbližší vyšší dvojnásobek n a čítač je globální index. Konečně, vygenerované čísla se transformují na 64 bitová čísla a převedou se do požadovaného intervalu $[0, 1)$, jak je ukázáno ve Zdrojovém kódu 4.17.

```
// Discretising
inline uint32
Discretise(const uint32 number, const uint32 max) {
    return (uint32)
        (((uint64) number * (uint64) max) >> 32);
}

// Continuing
inline uint64
Pack(const uint32 hi, const uint32 lo) {
    return ((uint64) hi << 32) | (uint64) lo;
}

const double INV2P53 = 1.0 / (double) (1UL << 53);

inline double
Continueise(const uint64 number) {
    const uint64 mantissa = number >> 11;
    return (double) mantissa * INV2P53;
}
```

Zdrojový kód 4.17: (GPU) Transformace vygenerované sekvence

4.2.7 Zápis vzorků do výstupního pole

Naměřené vzorky se zapíší do výstupního pole. V případě sekvenčního výpočtu se jednoduše projde pole vzorků a inkrementují se čítače na příslušných indexech výstupního pole. V případě paralelního výpočtu se naměřené vzorky seřadí a provede se paralelní redukce. Při redukci se seřazené naměřené vzorky kódují délkou běhu. Následně se kódované délky běhu sekvenčně zapíší do výstupního pole na příslušné indexy s očekáváním, že se ušetří mnoho náhodných přístupů do paměti.

Kapitola 5

Výsledky experimentů

Veškeré měření probíhalo na stolním počítači s následujícími specifikacemi:

- OS - Windows 10
- CPU - Intel i5-7600 (4 jádra)
- RAM - 16 GB
- GPU - Nvidia GeForce GTX 1060 6 GB

5.1 Robustnost Philox PRNG

Robustnost Philox PRNG byla měřena porovnáním distribucí naměřených výsledků pomocí Philox PRNG a Random.org. Celkem bylo naměřeno až 32M výsledků nad stavovým vektorem nad 10 qubity (16 KB). Stavový vektor byl uměle vygenerován podle několika známých pravděpodobnostních distribucí. Statistická podobnost distribucí naměřených výsledků byla měřena pomocí dvou metrik, a sice *celkovou variační vzdáleností* (TVD) a *Jensenovo-Shanonovo divergencí* (JSD).

Celková variační vzdálenost

Celková variační vzdálenost vyjadřuje maximální rozdíl mezi pravděpodobnostmi přiřazenými stejným událostem ve dvou porovnávaných rozděleních. Nabývá hodnot v intervalu $[0, 1]$, přičemž hodnota 0 značí totožné distribuce a vyšší hodnoty indikují rostoucí míru odlišnosti.

Jensenova–Shannonova divergence

Jensenova–Shannonova divergence je symetrická a vždy konečná míra rozdílu mezi dvěma pravděpodobnostními rozděleními, odvozená z Kullbackovy–Leiblerovy divergence. Na rozdíl od KL divergence je JSD numericky stabilní a omezená, přičemž hodnota 0 odpovídá identickým distribucím a rostoucí hodnota značí zvyšující se rozdíl mezi nimi.

Výsledky

Výsledky jsou shrnuty v tabulce A.1. Jejich průběh v závislosti na počtu vzorků je vizualizován v grafu A.1 a A.2. Pokud budeme předpokládat, že dobrá shoda nastává při $TVD < 0.05$ nebo $JSD < 0.01$, pak je potřeba alespoň 64-128K měření, aby se distribuce výsledků přiblížila té naměřené pomocí skutečné náhody.

5.2 Urychlení výpočtů

Výpočetní čas

Čas výpočtů byl měřen jako průměr 100 opakovaných měření jednotlivých operací po 10 zahřívacích opakováních. V případě výpočtu na GPU se jedná o výpočetní čas samotného kernelu. Jednotlivé časy výpočtů jsou uvedeny v příloze v tabulkách C.1, C.2, C.3, C.4 a C.5.

Metriky jednotlivých operací

Urychlení jednotlivých operací jsou uvedeny v tabulce B.1 a B.2. Jejich průběh je vizualizován v grafech B.1, B.2, B.3 a B.4.

Výpočet pravděpodobností

Výpočet pravděpodobností dosáhl urychlení 1.40, respektive 7.76 a paralelizace urychlila výpočet za předpokladu, že se počítá alespoň nad stavovým vektorem o 32K stavech.

Generování pseudonáhodných diskrétních čísel

Generování pseudonáhodných diskrétních čísel dosáhlo urychlení 3.99, respektive 77.58 a paralelizace urychlila výpočet za předpokladu, že se generuje alespoň 4K čísel.

Generování pseudonáhodných spojitých čísel

Generování pseudonáhodných spojitých čísel dosáhlo urchlení 3.89, respektive 137.80 a paralelizace urychlila výpočet za předpokladu, že se generuje alespoň 2K čísel.

Vzorkování z Alias tabulky

Vzorkování z Alias tabulky dosáhlo urchlení 2.05, respektive 14.11 a paralelizace urychlila výpočet za předpokladu, že se vzorkuje alespoň 8K čísel.

Zápis vzorků do výstupního pole

Zápis vzorků do výstupního pole dosáhl urchlení 3.39 a paralelizace urychlila výpočet za předpokladu, že bylo zapsáno alespoň 1M vzorků.

Metriky celého programu

Výpočetní čas

Výpočetní čas celého procesu simulace měření je 14 599 ms v případě sekvencního skalárního výpočtu, 10 837 ms v případě vícevláknového výpočtu s vektorizací a 8 300 ms v případě paralelního výpočtu s využitím GPU.

Urychlení

Celkové urchlení vícevláknovým výpočtem s vektorizací dosahuje $S_{par} = 1.35$ a celkový výpočetní čas se zkrátil o 25.77 %. V případě paralelního výpočtu s využitím GPU dosahuje celkové zrychlení $S_{gpu} = 1.76$ a celkový výpočetní čas se zkrátil o 43.14 %.

Karp-Flattova metrika

Karp-Flattova metrika nabývá hodnoty $e_{par} = 0.73$ v případě paralelního výpočtu s vektorizací, za předpokladu $p = 16$, tj. 4 jádra a 4 linky SIMD vektoru pro převážně používaný datový typ double. V případě paralelního výpočtu s využitím GPU nabývá Karp-Flattova metrika hodnoty $e_{gpu} = 0.57$, za předpokladu $p = 1280$ CUDA jader.

5.3 Poznámky

Nízké urchlení několika částí výpočtu i vysoká Karp-Flattova metrika celého procesu měření může být vysvětlena primárně tím, že nezanedbatelná část

výpočtu je velmi málo aritmeticky náročná a je výrazně paměťově vázaná, jako např. odstranění prokládání, výpočet pravděpodobnosti nebo vzorkování z pravděpodobnostní distribuce. Dále, samotné sestavení vzorkovací tabulky je neparalelizovatelné a časově netriviálně náročné. V průběhu výpočtu je také potřeba alokovat velké množství paměti. V případě paralelního výpočtu na GPU se navíc započítává režie přesunu dat na GPU a zpět.

Nezanedbatelného zlepšení jsem dosáhl, když jsem při odstraňování prokládání ve vstupních datech a při výpočtu (zápisu) pravděpodobností využil instrukci `_mm256_stream_pd` místo `_mm256_store_pd`. Při použití této instrukce se obchází cache procesoru a nedochází ke *cache polluting*.

Kapitola 6

Uživatelská příručka

Řešení vyžaduje překladač MSVC jazyka C++ podporující standard C++20, překladač FPC jazyka Pascal (Object Pascal), GPU a potřebné ovladače OpenCL.

6.1 Překlad a sestavení

Po zadání příkazu:

```
$ Setup.bat
```

dávkový soubor nastaví prostředí pro překlad aplikace a stáhne potřebné knihovny pro běh aplikace (OpenCL, oneTBB).

Po zadání příkazu:

```
$ Build.bat [<ExecutionPolicy> [<PrngAlgorithm>]]
```

dávkový soubor přeloží simulátor kvantového počítače, knihovnu pro simulaci měření stavu kvantového výpočtu, připojí závislosti a sestaví celkovou aplikaci s demonstračním kvantovým obvodem. Výsledný spustitelný soubor, dynamicky linkovaná knihovna a další závislosti se nachází v adresáři `bin`.

ExecutionPolicy

- Sequential - sekvenční skalární
- Parallel - vícevláknový s manuální vektorizací
- Accelerated - paralelní s využitím GPU

PrngAlgorithm

- Philox - vlastní implementace Philox

- MT19937 - knihovní implementace Mersenne Twister
- RandomOrg - archiv skutečně náhodných čísel

6.2 Spuštění

Demonstrační aplikace se spustí po zadání příkazu:

```
$ Run.bat
```

Kapitola 7

Závěr

Cílem této práce bylo navrhnout a implementovat simulaci měření kvantového stavu s důrazem na efektivní paralelní zpracování. Byla realizována knihovní implementace ve třech variantách: sekvenční skalární, vícevláknová s manuální vektorizací pomocí instrukcí AVX2 a paralelní s využitím GPU prostřednictvím OpenCL. Řešení bylo integrováno jako dynamicky linkovaná knihovna k existujícímu simulátoru kvantového počítače. Navržené řešení splňuje stanovené cíle práce.

V rámci práce byl navržen a implementován efektivní proces simulace měření. Statistická kvalita generovaných náhodných hodnot byla ověřena porovnáním s daty ze služby Random.org. Výsledky ukázaly, že při dostatečném počtu měření dosahuje Philox PRNG velmi dobré shody se skutečně náhodným zdrojem, přičemž metriky celkové variační vzdálenosti a Jensenovy–Shannonovy divergence s rostoucím počtem vzorků konvergují k nízkým hodnotám.

Měření ověřila, že paralelizace výpočtu vedou k výraznému zkrácení výpočetního času, zejména u generování pseudonáhodných čísel. Zároveň se ukázalo, že některé části výpočtu jsou výrazně paměťově vázané nebo obtížně paralelizovatelné, což omezuje dosažitelné urychlení.

Bibliografie

- [1] M. Vose, “A linear algorithm for generating random numbers with a given distribution,” *IEEE Transactions on Software Engineering*, roč. 17, č. 9, s. 972–975, 1991.
- [2] M. Haahr, *RANDOM.ORG: True Random Number Service*, <https://www.random.org/>, Accessed: 2026-01-08, 2026.
- [3] J. K. Salmon, M. A. Moraes, R. O. Dror a D. E. Shaw, “Parallel Random Numbers: As Easy as 1, 2, 3,” in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis (SC11)*, Seattle, Washington, USA: ACM, 2011, s. 1–12.

Seznam obrázků

3.1	Diagram řešení	7
A.1	Celková variační vzdálenost	34
A.2	Jensenova–Shannonova divergence	35
B.1	Výpočet pravděpodobností	38
B.2	Generování diskrétních pseudonáhodných čísel	38
B.3	Generování spojitých pseudonáhodných čísel	39
B.4	Vzorkování z pravděpodobnostní distribuce	39

Seznam tabulek

A.1	Statistická podobnost distribucí naměřených výsledků	33
B.1	Urychlení výpočetního času	36
B.2	Urychlení výpočetního času	37
C.1	Výpočet pravděpodobností	40
C.2	Generování diskrétních pseudonáhodných čísel	41
C.3	Generování spojitých pseudonáhodných čísel	42
C.4	Vzorkování z Alias tabulky	43
C.5	Zápis vzorků do výstupního pole	44

Seznam zdrojových kódů

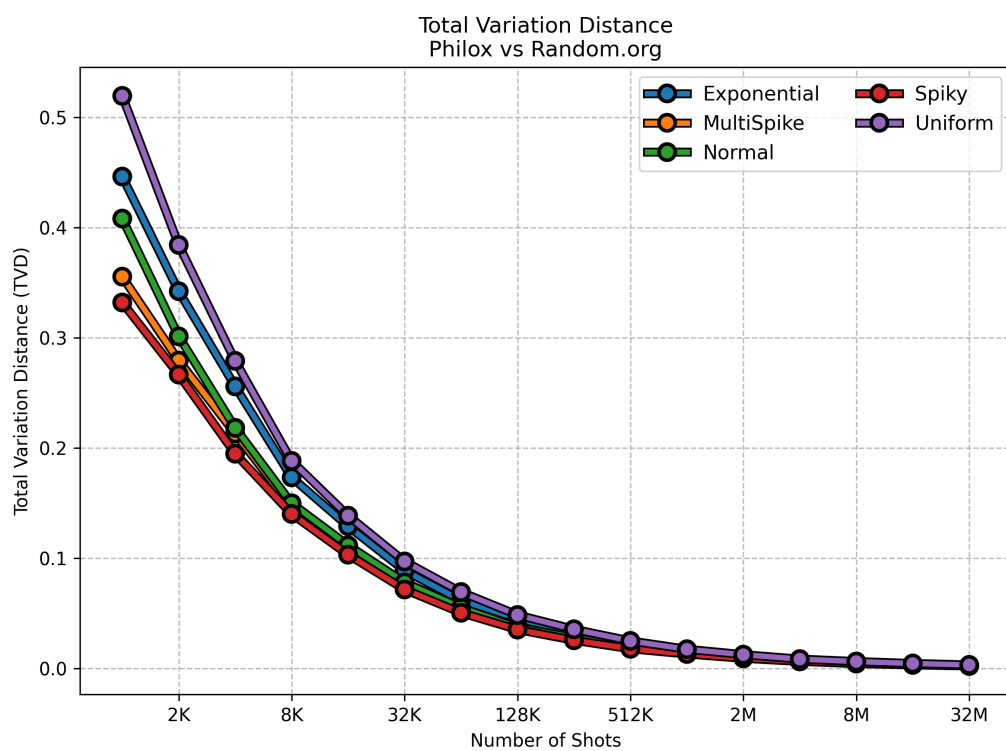
4.1	Dynamická alokace zarovnané paměti	9
4.2	Výčet způsobů výpočtu	10
4.3	Výčet generátorů náhodných čísel	10
4.4	Kontejner specifický pro výpočetní zařízení	11
4.5	Ukázka rozhraní CLManager	11
4.6	Rozhraní vstupního bodu programu	12
4.7	Běh simulace	12
4.8	(Par) Odstranění prokládání	13
4.9	(Seq) Výpočet pravděpodobností	14
4.10	(Par) Výpočet pravděpodobností	14
4.11	(GPU) Výpočet pravděpodobností	15
4.12	Vzorkování z Alias tabulky	16
4.13	(Seq) Vzorkování z Alias tabulky	17
4.14	(Par) Vzorkování z Alias tabulky	17
4.15	(GPU) Vzorkování z Alias tabulky	17
4.16	(GPU) Philox PRNG	19
4.17	(GPU) Transformace vygenerované sekvence	20

Příloha A

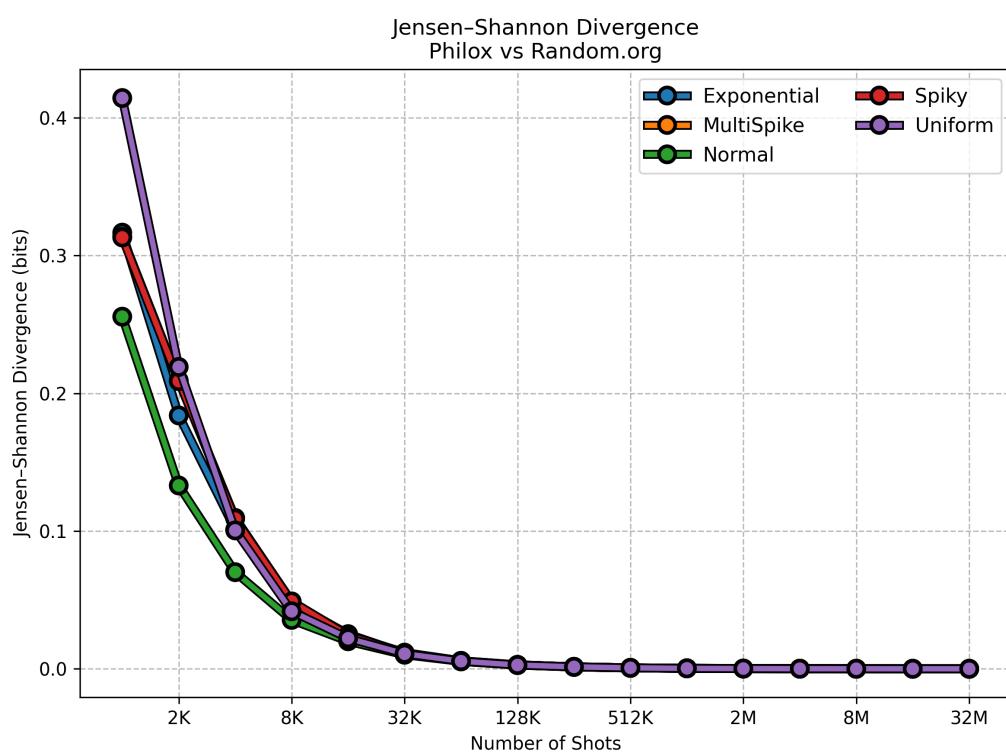
Statistická podobnost distribucí naměřených výsledků

Tabulka A.1: Statistická podobnost distribucí naměřených výsledků

Shots	Distribuce									
	Uniform		Normal		Spiky		MultiSpike		Exponential	
	TVD	JSD	TVD	JSD	TVD	JSD	TVD	JSD	TVD	JSD
1K	0.520	0.414	0.408	0.256	0.332	0.313	0.355	0.314	0.446	0.317
2K	0.384	0.219	0.301	0.133	0.267	0.209	0.279	0.209	0.342	0.184
4K	0.279	0.100	0.218	0.070	0.195	0.109	0.213	0.110	0.256	0.101
8K	0.188	0.042	0.150	0.035	0.140	0.049	0.147	0.049	0.173	0.048
16K	0.139	0.022	0.112	0.020	0.103	0.024	0.111	0.024	0.129	0.025
32K	0.097	0.011	0.078	0.010	0.072	0.012	0.077	0.012	0.089	0.011
64K	0.069	0.006	0.055	0.005	0.050	0.006	0.053	0.006	0.062	0.005
128K	0.048	0.003	0.037	0.003	0.035	0.003	0.038	0.003	0.045	0.003
256K	0.035	0.001	0.028	0.001	0.026	0.001	0.027	0.001	0.032	0.001
512K	0.025	0.001	0.019	0.001	0.018	0.001	0.019	0.001	0.022	0.001
1M	0.017	0.000	0.014	0.000	0.013	0.000	0.014	0.000	0.016	0.000
2M	0.012	0.000	0.010	0.000	0.009	0.000	0.010	0.000	0.011	0.000
4M	0.008	0.000	0.007	0.000	0.006	0.000	0.007	0.000	0.008	0.000
8M	0.006	0.000	0.005	0.000	0.004	0.000	0.005	0.000	0.006	0.000
16M	0.004	0.000	0.003	0.000	0.003	0.000	0.003	0.000	0.004	0.000
32M	0.003	0.000	0.002	0.000	0.002	0.000	0.002	0.000	0.003	0.000



Obrázek A.1: Celková variační vzdálenost



Obrázek A.2: Jensenova-Shannonova divergence

Příloha B

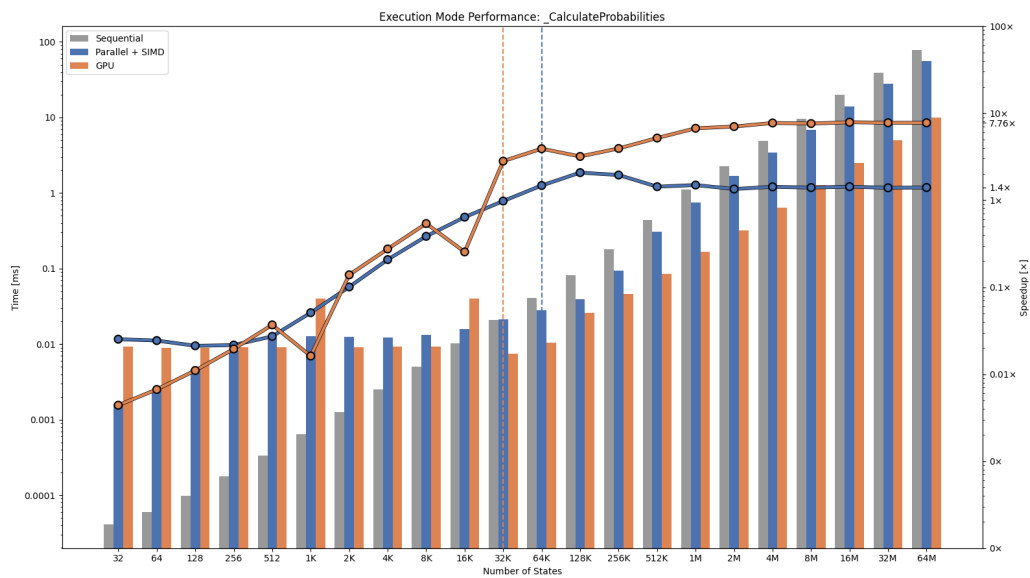
Urychlení výpočetního času

Tabulka B.1: Urychlení výpočetního času

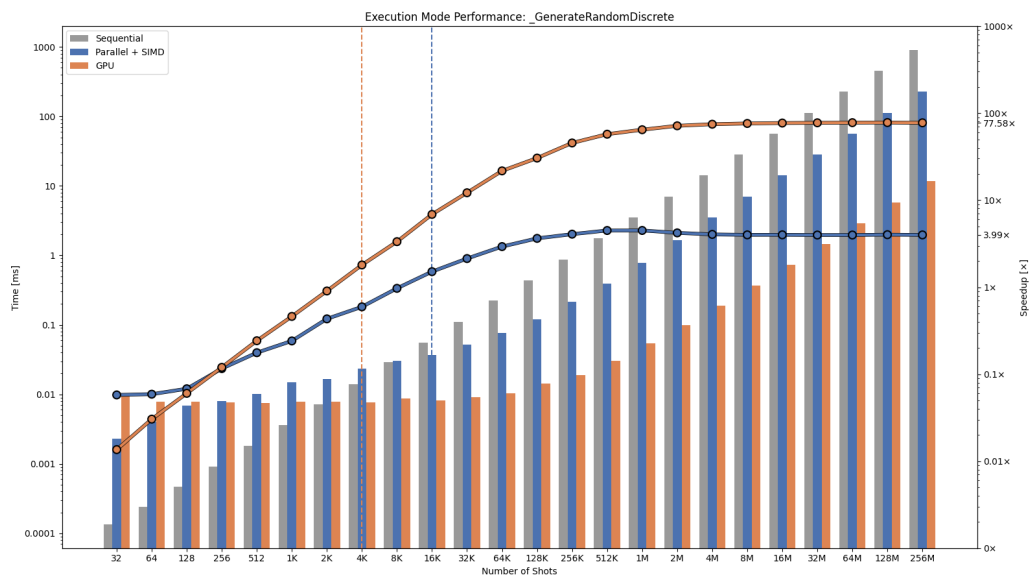
	Funkce	
States	Calc.Prob.	
	Par	GPU
32	0.03	0.00
64	0.02	0.01
128	0.02	0.01
256	0.02	0.02
512	0.03	0.04
1K	0.05	0.02
2K	0.10	0.14
4K	0.21	0.28
8K	0.38	0.54
16K	0.64	0.25
32K	0.98	2.81
64K	1.48	3.91
128K	2.08	3.19
256K	1.95	3.93
512K	1.43	5.18
1M	1.49	6.71
2M	1.35	7.03
4M	1.43	7.72
8M	1.40	7.61
16M	1.43	7.88
32M	1.39	7.76
64M	1.40	7.76

Tabulka B.2: Urychlení výpočetního času

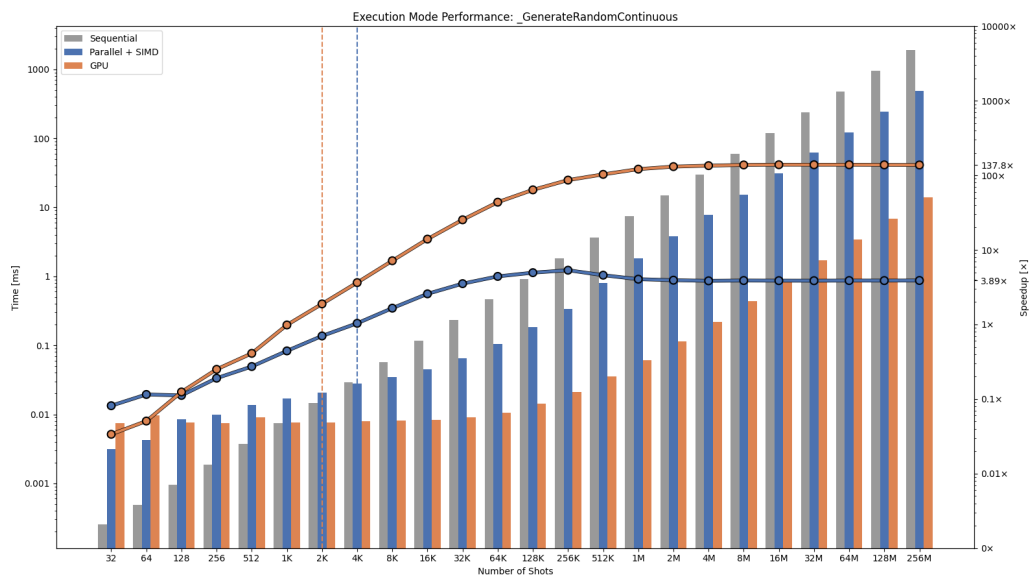
	Funkce						
Shots	Gen.R.Disc.		Gen.R.Cont.		Samp.A.Tab.		FlushSamp.
	Par	GPU	Par	GPU	Par	GPU	Par
32	0.06	0.01	0.08	0.03	0.04	0.01	0.01
64	0.06	0.03	0.12	0.05	0.03	0.01	0.01
128	0.07	0.06	0.11	0.12	0.04	0.02	0.02
256	0.12	0.12	0.19	0.25	0.05	0.03	0.07
512	0.18	0.24	0.27	0.41	0.06	0.06	0.07
1K	0.24	0.46	0.44	0.99	0.11	0.13	0.13
2K	0.43	0.91	0.70	1.89	0.15	0.26	0.34
4K	0.60	1.80	1.04	3.64	0.32	0.57	0.58
8K	0.97	3.34	1.66	7.16	0.54	1.20	0.73
16K	1.51	6.87	2.58	13.97	0.88	2.48	0.79
32K	2.13	12.18	3.53	25.36	1.41	4.36	0.78
64K	2.93	21.75	4.42	43.68	1.67	7.79	0.35
128K	3.64	30.50	4.95	63.88	1.93	7.96	0.53
256K	4.07	45.52	5.35	86.39	2.10	9.60	0.54
512K	4.49	57.38	4.58	103.10	2.28	11.81	0.72
1M	4.49	64.62	4.05	121.12	2.24	12.89	1.04
2M	4.21	71.70	3.93	131.05	2.15	13.54	1.58
4M	4.05	74.74	3.85	134.94	2.11	13.86	2.00
8M	4.00	76.28	3.89	137.98	2.08	13.95	2.49
16M	4.00	77.11	3.87	138.92	2.06	13.99	2.93
32M	3.98	77.61	3.87	138.77	2.05	14.08	3.17
64M	3.98	77.87	3.88	138.75	2.06	14.08	3.47
128M	4.02	78.07	3.88	138.29	2.05	14.12	3.37
256M	3.99	77.58	3.89	137.80	2.05	14.11	3.39



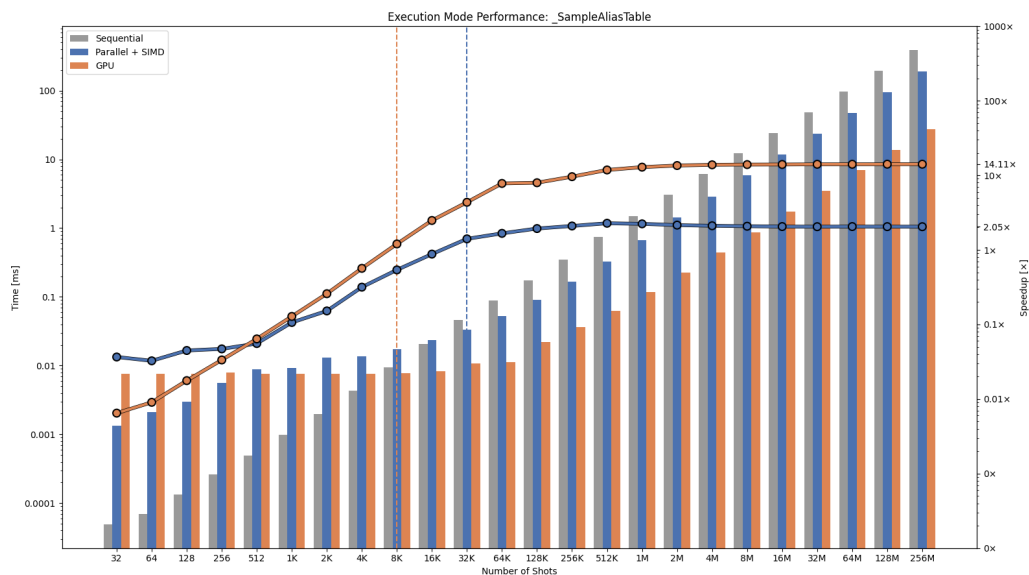
Obrázek B.1: Výpočet pravděpodobností



Obrázek B.2: Generování diskrétních pseudonáhodných čísel



Obrázek B.3: Generování spojitých pseudonáhodných čísel



Obrázek B.4: Vzorkování z pravděpodobnostní distribuce

Příloha C

Naměřený výpočetní čas

Tabulka C.1: Výpočet pravděpodobností

	Calc.Prob.		
States	Výpočetní čas [ms]		
	Seq	Par	GPU
32	0.000041	0.001623	0.009308
64	0.000060	0.002463	0.008986
128	0.000099	0.004666	0.008972
256	0.000178	0.008237	0.009127
512	0.000335	0.012305	0.009078
1K	0.000646	0.012683	0.039891
2K	0.001268	0.012584	0.009138
4K	0.002539	0.012226	0.009231
8K	0.005050	0.013134	0.009363
16K	0.010179	0.015927	0.039995
32K	0.020986	0.021460	0.007458
64K	0.041120	0.027869	0.010523
128K	0.081969	0.039481	0.025732
256K	0.181240	0.093056	0.046073
512K	0.441756	0.308614	0.085220
1M	1.115910	0.746899	0.166251
2M	2.257240	1.677790	0.321277
4M	4.909440	3.442650	0.635623
8M	9.627670	6.890000	1.265670
16M	19.870200	13.878300	2.522660
32M	39.146400	28.184000	5.041530
64M	78.207500	55.859000	10.076400

Tabulka C.2: Generování diskretních pseudonáhodných čísel

	Gen.R.Disc.		
Shots	Výpočetní čas [ms]		
	Seq	Par	GPU
32	0.000134	0.002320	0.009860
64	0.000239	0.004057	0.007852
128	0.000471	0.006924	0.007815
256	0.000919	0.007924	0.007665
512	0.001807	0.010227	0.007453
1K	0.003589	0.014877	0.007777
2K	0.007126	0.016475	0.007873
4K	0.013931	0.023409	0.007732
8K	0.029213	0.030106	0.008749
16K	0.055942	0.037054	0.008141
32K	0.110957	0.051979	0.009109
64K	0.223476	0.076274	0.010276
128K	0.439636	0.120733	0.014412
256K	0.867153	0.212984	0.019049
512K	1.751570	0.390407	0.030524
1M	3.501000	0.779367	0.054175
2M	7.040110	1.671310	0.098192
4M	14.102800	3.485440	0.188694
8M	28.227500	7.062420	0.370034
16M	56.351900	14.084800	0.730823
32M	112.847000	28.349900	1.453950
64M	225.829000	56.720600	2.900170
128M	452.557000	112.685000	5.796750
256M	903.987000	226.435000	11.652500

Tabulka C.3: Generování spojitých pseudonáhodných čísel

	Gen.R.Cont.		
Shots	Výpočetní čas [ms]		
	Seq	Par	GPU
32	0.000254	0.003121	0.007541
64	0.000486	0.004219	0.009588
128	0.000946	0.008455	0.007598
256	0.001877	0.009853	0.007500
512	0.003741	0.013702	0.009122
1K	0.007443	0.016824	0.007556
2K	0.014512	0.020679	0.007683
4K	0.029016	0.027966	0.007961
8K	0.057751	0.034878	0.008064
16K	0.116127	0.045070	0.008313
32K	0.231327	0.065556	0.009123
64K	0.463421	0.104944	0.010611
128K	0.904193	0.182557	0.014154
256K	1.810750	0.338763	0.020961
512K	3.663910	0.799582	0.035537
1M	7.413610	1.830420	0.061209
2M	14.950700	3.807740	0.114082
4M	29.821400	7.739500	0.220994
8M	59.701900	15.354400	0.432681
16M	119.404000	30.819000	0.859492
32M	239.356000	61.852300	1.724890
64M	477.684000	122.982000	3.442690
128M	953.716000	245.706000	6.896560
256M	1908.440000	490.220000	13.849800

Tabulka C.4: Vzorkování z Alias tabulky

	Samp.A.Tab.		
Shots	Výpočetní čas [ms]		
	Seq	Par	GPU
32	0.000049	0.001337	0.007564
64	0.000069	0.002118	0.007629
128	0.000134	0.002991	0.007582
256	0.000263	0.005589	0.007866
512	0.000487	0.008764	0.007601
1K	0.000976	0.009169	0.007609
2K	0.001972	0.012969	0.007623
4K	0.004300	0.013571	0.007593
8K	0.009375	0.017371	0.007796
16K	0.020486	0.023302	0.008253
32K	0.046646	0.033143	0.010711
64K	0.088266	0.052937	0.011329
128K	0.174384	0.090146	0.021907
256K	0.350112	0.167112	0.036461
512K	0.743255	0.326079	0.062935
1M	1.504240	0.672160	0.116714
2M	3.048200	1.417140	0.225063
4M	6.096860	2.891550	0.440037
8M	12.170700	5.855740	0.872601
16M	24.346200	11.835100	1.739700
32M	48.808500	23.816400	3.466950
64M	97.595700	47.449500	6.930550
128M	195.431000	95.245100	13.845100
256M	390.300000	190.544000	27.666300

Tabulka C.5: Zápis vzorků do výstupního pole

	FlushSamp.	
Shots	Výpočetní čas [ms]	
	Seq	Par
32	0.000095	0.009905
64	0.000225	0.015145
128	0.000500	0.022570
256	0.001620	0.022320
512	0.002110	0.028500
1K	0.004825	0.036165
2K	0.017455	0.052000
4K	0.047590	0.081815
8K	0.095885	0.131595
16K	0.190625	0.241010
32K	0.387860	0.499505
64K	0.728835	2.069440
128K	1.727040	3.228720
256K	3.209420	5.889760
512K	5.652920	7.801230
1M	11.217900	10.780500
2M	22.725700	14.384900
4M	44.973400	22.530600
8M	89.868700	36.092700
16M	180.155000	61.581800
32M	361.049000	113.750000
64M	755.887000	217.954000
128M	1451.690000	430.276000
256M	2898.810000	854.955000